

Data Structures Notes: Doubly Linked List and Circular Linked List

1. Doubly Linked List

A **doubly linked list (DLL)** is a linked list in which each node contains **three fields**:

1. **Previous** (`prev`) – address of the previous node
2. **Data** – actual data
3. **Next** (`next`) – address of the next node

Structure of a node

```
+-----+-----+-----+
|  PREV  |  DATA  |  NEXT  |
+-----+-----+-----+
```

For example:

```
NULL ← [10] ⇌ [20] ⇌ [30] → NULL
        ↑           ↑
      HEAD       TAIL
```

The first node has:

```
prev = NULL
```

The last node has:

```
next = NULL
```

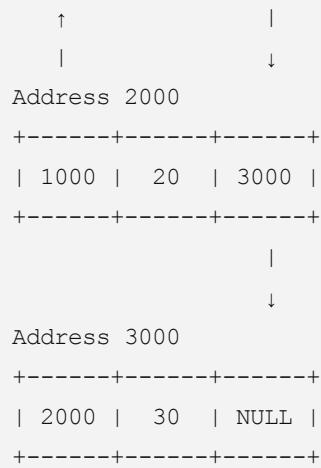
2. Representation of Doubly Linked List

Consider:

```
10 ⇌ 20 ⇌ 30
```

Memory representation can be:

```
Address 1000
+-----+-----+-----+
| NULL | 10 | 2000 |
+-----+-----+-----+
```



Each node contains links in **both directions**.

Therefore, traversal is possible:

```
Forward:
10 → 20 → 30

Backward:
30 → 20 → 10
```

3. Node Declaration in C

```
C

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
```

Pointers:

```
C

struct Node *head = NULL;
struct Node *tail = NULL;
```

4. Advantages of Doubly Linked List

- Can be traversed in both directions.

- Deletion of a known node is easier.
- Insertion before a given node is easier.
- Useful for navigation systems.
- Useful for undo/redo operations.
- Useful for browser history.

Disadvantages

- Requires extra memory for `prev`.
- More pointer manipulation is required.
- Implementation is more complicated than a singly linked list.

5. Operations on Doubly Linked List

The major operations are:

1. Traversal
2. Reverse traversal
3. Searching
4. Insertion at beginning
5. Insertion at end
6. Insertion before a node
7. Insertion after a node
8. Deletion from beginning
9. Deletion from end
10. Deletion of a specific node

6. Traversal of Doubly Linked List

Suppose:

HEAD

↓

10 ⇌ 20 ⇌ 30 ⇌ 40

Algorithm: Forward Traversal

FORWARD_TRAVERSE(HEAD)

1. Start.
2. Set `PTR = HEAD`.

3. Repeat while `PTR != NULL` :
 - Display `PTR->DATA` .
 - Set `PTR = PTR->NEXT` .
4. Stop.

C code

```
C

void forwardTraversal(struct Node *head)
{
    struct Node *ptr = head;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```

Complexity

Time = $O(n)$

Space = $O(1)$

7. Reverse Traversal

Reverse traversal starts from the last node.

```
TAIL
↓
40 ⇌ 30 ⇌ 20 ⇌ 10
```

Algorithm

REVERSE_TRAVERSE(TAIL)

1. Set `PTR = TAIL` .
2. Repeat while `PTR != NULL` :
 - Display `PTR->DATA` .
 - Set `PTR = PTR->PREV` .
3. Stop.

C code

C

```
void reverseTraversal(struct Node *tail)
{
    struct Node *ptr = tail;

    while (ptr != NULL)
    {
        printf("%d ", ptr->data);
        ptr = ptr->prev;
    }
}
```

Complexity

Time = $O(n)$

Space = $O(1)$

8. Searching in Doubly Linked List

Suppose:

10 ⇌ 20 ⇌ 30 ⇌ 40

Search for 30 .

10 → No
20 → No
30 → Found

Algorithm

SEARCH(HEAD, ITEM)

1. Set PTR = HEAD .
2. Repeat while PTR != NULL :
 - If PTR->DATA == ITEM , return FOUND.
 - Set PTR = PTR->NEXT .
3. Return NOT FOUND.
4. Stop.

Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

9. Insertion at Beginning

Original:

```

HEAD
↓
10 ⇌ 20 ⇌ 30

```

Insert 5 .

```

HEAD
↓
5 ⇌ 10 ⇌ 20 ⇌ 30

```

Algorithm

INSERT_BEGINNING(HEAD, ITEM)

1. Create a new node `NEW` .
2. Store `ITEM` in `NEW->DATA` .
3. Set `NEW->PREV = NULL` .
4. Set `NEW->NEXT = HEAD` .
5. If `HEAD != NULL` , set `HEAD->PREV = NEW` .
6. Set `HEAD = NEW` .
7. If the list was empty, set `TAIL = NEW` .
8. Stop.

Complexity

Time = $O(1)$

10. Insertion at End

Original:

10 ⇌ 20 ⇌ 30

Insert 40 .

10 ⇌ 20 ⇌ 30 ⇌ 40

If a `TAIL` pointer is maintained, insertion is very efficient.

Algorithm

INSERT_END(TAIL, ITEM)

1. Create `NEW` .
2. Set `NEW->DATA = ITEM` .
3. Set `NEW->NEXT = NULL` .
4. Set `NEW->PREV = TAIL` .
5. If `TAIL != NULL` , set `TAIL->NEXT = NEW` .
6. Set `TAIL = NEW` .
7. If list was empty, set `HEAD = NEW` .
8. Stop.

Complexity

With `TAIL` pointer:

Time = $O(1)$

Without `TAIL` :

Time = $O(n)$

11. Insertion After a Given Node

Suppose:

10 ⇌ 20 ⇌ 40

Insert 30 after 20 .

Result:

10 ⇄ 20 ⇄ 30 ⇄ 40

Pointer changes

Before:

20 ⇄ 40

After:

20 ⇄ 30 ⇄ 40

Algorithm

INSERT_AFTER(PTR, ITEM)

1. Create `NEW` .
2. Set `NEW->DATA = ITEM` .
3. Set `NEW->NEXT = PTR->NEXT` .
4. Set `NEW->PREV = PTR` .
5. If `PTR->NEXT != NULL` , set `PTR->NEXT->PREV = NEW` .
6. Set `PTR->NEXT = NEW` .
7. If `PTR` was the last node, update `TAIL` .
8. Stop.

Complexity

If `PTR` is already known:

O(1)

If the node has to be searched:

O(n)

12. Insertion Before a Given Node

Suppose:

10 ⇄ 30 ⇄ 40

Insert `20` before `30` .

Result:

10 ⇌ 20 ⇌ 30 ⇌ 40

Algorithm

INSERT_BEFORE(PTR, ITEM)

1. Create `NEW` .
2. Set `NEW->DATA = ITEM` .
3. Set `NEW->PREV = PTR->PREV` .
4. Set `NEW->NEXT = PTR` .
5. If `PTR->PREV != NULL` , set `PTR->PREV->NEXT = NEW` .
6. Otherwise set `HEAD = NEW` .
7. Set `PTR->PREV = NEW` .
8. Stop.

Complexity

If `PTR` is known:

O(1)

13. Deletion from Beginning

Original:

HEAD
↓
10 ⇌ 20 ⇌ 30

Delete 10 .

HEAD
↓
20 ⇌ 30

Algorithm

DELETE_BEGINNING(HEAD)

1. If `HEAD == NULL` , report **UNDERFLOW** .
2. Set `TEMP = HEAD` .

3. Set `HEAD = HEAD->NEXT` .
4. If `HEAD != NULL` , set `HEAD->PREV = NULL` .
5. Otherwise set `TAIL = NULL` .
6. Free `TEMP` .
7. Stop.

Complexity

O(1)

14. Deletion from End

Original:

10 ⇌ 20 ⇌ 30

Delete 30 .

10 ⇌ 20

Algorithm

DELETE_END(TAIL)

1. If `TAIL == NULL` , report underflow.
2. Set `TEMP = TAIL` .
3. Set `TAIL = TAIL->PREV` .
4. If `TAIL != NULL` , set `TAIL->NEXT = NULL` .
5. Otherwise set `HEAD = NULL` .
6. Free `TEMP` .
7. Stop.

Complexity

With `TAIL` :

O(1)

Without `TAIL` :

O(n)

15. Deletion of a Specific Node

Suppose:

10 ⇌ 20 ⇌ 30 ⇌ 40

Delete 30 .

Result:

10 ⇌ 20 ⇌ 40

Algorithm

DELETE_NODE(PTR)

1. If `PTR == NULL` , stop.
2. If `PTR->PREV != NULL` :
 - `PTR->PREV->NEXT = PTR->NEXT`
3. Else:
 - `HEAD = PTR->NEXT`
4. If `PTR->NEXT != NULL` :
 - `PTR->NEXT->PREV = PTR->PREV`
5. Else:
 - `TAIL = PTR->PREV`
6. Free `PTR` .
7. Stop.

Complexity

If the node is already known:

O(1)

If the node must first be searched:

O(n)

16. Complexity of Doubly Linked List

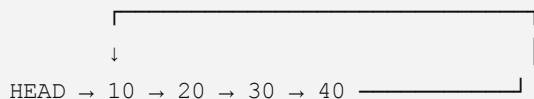
Operation	Complexity
Forward traversal	O(n)

Operation	Complexity
Reverse traversal	O(n)
Search	O(n)
Insert at beginning	O(1)
Insert at end with TAIL	O(1)
Insert after known node	O(1)
Insert before known node	O(1)
Delete beginning	O(1)
Delete end with TAIL	O(1)
Delete known node	O(1)
Delete searched node	O(n)

17. Circular Linked List

A **circular linked list** is a linked list in which the last node points back to the first node instead of pointing to `NULL`.

Singly Circular Linked List



The last node satisfies:

C

```
last->next = head;
```

Therefore:

There is **no NULL link** at the end of a circular linked list.

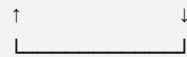
18. Difference Between Linear and Circular Linked List

Linear linked list

10 → 20 → 30 → NULL

Circular linked list

10 → 20 → 30



19. Representation of Circular Linked List

A node can be defined as:

C

```
struct Node
{
    int data;
    struct Node *next;
};
```

We can maintain:

C

```
struct Node *head = NULL;
struct Node *tail = NULL;
```

For a non-empty circular list:

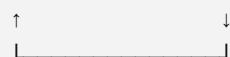
tail->next = head

Example:

HEAD

↓

10 → 20 → 30 → 40

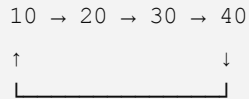


TAIL

20. Traversal of Circular Linked List

Traversal is different because there is no `NULL` to indicate the end.

For example:



We stop when we reach `HEAD` again.

Algorithm

TRAVERSE(HEAD)

1. If `HEAD == NULL`, list is empty.
2. Set `PTR = HEAD`.
3. Repeat:
 - Display `PTR->DATA`.
 - `PTR = PTR->NEXT`.
4. Continue while `PTR != HEAD`.
5. Stop.

C code

C

```
void display(struct Node *head)
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    do
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    while (temp != head);
```

```
printf("(HEAD)\n");  
}
```

Complexity

Time = $O(n)$

Space = $O(1)$

21. Searching in Circular Linked List

Algorithm

SEARCH(HEAD, ITEM)

1. If `HEAD == NULL`, return NOT FOUND.
2. Set `PTR = HEAD`.
3. Repeat:
 - If `PTR->DATA == ITEM`, return FOUND.
 - Set `PTR = PTR->NEXT`.
4. Continue while `PTR != HEAD`.
5. Return NOT FOUND.

Complexity

Best case:

$O(1)$

Worst case:

$O(n)$

22. Insertion at Beginning in Circular Linked List

Suppose:

```
10 → 20 → 30  
↑         ↓  
└────────┘
```

Insert 5.

Result:

5 → 10 → 20 → 30



If both **HEAD** and **TAIL** are maintained:

Algorithm

1. Create **NEW**.
2. If list is empty:
 - **HEAD = TAIL = NEW**
 - **NEW->NEXT = HEAD**
3. Otherwise:
 - **NEW->NEXT = HEAD**
 - **HEAD = NEW**
 - **TAIL->NEXT = HEAD**
4. Stop.

C code

C

```
void insertBeginning(struct Node **head,
                    struct Node **tail,
                    int value)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
        return;

    newNode->data = value;

    if (*head == NULL)
    {
        *head = *tail = newNode;
        newNode->next = newNode;
    }
    else
    {
        newNode->next = *head;
        *head = newNode;
        (*tail)->next = *head;
    }
}
```

Complexity

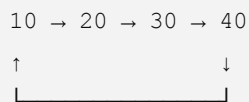
$O(1)$

23. Insertion at End

Suppose:



Insert 40 .



Algorithm

1. Create `NEW` .
2. Set `NEW->DATA = ITEM` .
3. If list is empty:
 - `HEAD = TAIL = NEW`
 - `NEW->NEXT = HEAD`
4. Otherwise:
 - `NEW->NEXT = HEAD`
 - `TAIL->NEXT = NEW`
 - `TAIL = NEW`
5. Stop.

Complexity

With `TAIL` pointer:

$O(1)$

Without `TAIL` pointer:

$O(n)$

24. Insertion After a Given Node

Suppose:

10 → 20 → 40 → back to 10

Insert 30 after 20 .

Result:

10 → 20 → 30 → 40 → back to 10

Algorithm

1. Search for the given node.
2. Create NEW .
3. Set NEW->NEXT = PTR->NEXT .
4. Set PTR->NEXT = NEW .
5. If PTR == TAIL , update TAIL = NEW .
6. Stop.

Complexity

Search required:

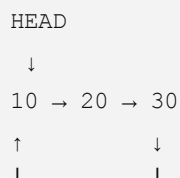
O(n)

If node is already known:

O(1)

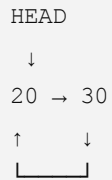
25. Deletion from Beginning

Original:



Delete 10 .

Result:



Algorithm

1. If `HEAD == NULL`, report underflow.
2. If `HEAD == TAIL`:
 - Free `HEAD`.
 - Set `HEAD = TAIL = NULL`.
3. Otherwise:
 - `TEMP = HEAD`
 - `HEAD = HEAD->NEXT`
 - `TAIL->NEXT = HEAD`
 - Free `TEMP`.
4. Stop.

Complexity

$O(1)$

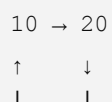
26. Deletion from End

Original:



Delete `30`.

Result:



Since a singly circular linked list has no `prev` pointer, we must find the node before `TAIL`.

Algorithm

1. If list is empty, report underflow.

2. If `HEAD == TAIL` :
 - Free `HEAD` .
 - Set `HEAD = TAIL = NULL` .
3. Otherwise:
 - Set `PTR = HEAD` .
 - Move until `PTR->NEXT == TAIL` .
 - Set `PTR->NEXT = HEAD` .
 - Free `TAIL` .
 - Set `TAIL = PTR` .
4. Stop.

Complexity

$O(n)$

27. Deletion of a Specific Element

Suppose:

10 → 20 → 30 → 40 → back to 10

Delete 30 .

Result:

10 → 20 → 40 → back to 10

Algorithm

DELETE_VALUE(HEAD, ITEM)

1. If list is empty, report underflow.
2. Set `CURRENT = HEAD` .
3. Set `PREVIOUS = TAIL` .
4. Repeat:
 - If `CURRENT->DATA == ITEM` , go to step 5.
 - `PREVIOUS = CURRENT`
 - `CURRENT = CURRENT->NEXT`
5. If `CURRENT == HEAD` and it was not found, report NOT FOUND.
6. Set `PREVIOUS->NEXT = CURRENT->NEXT` .
7. If `CURRENT == HEAD` , update `HEAD` .

8. If `CURRENT == TAIL` , update `TAIL` .
9. Free `CURRENT` .
10. Stop.

Complexity

$O(n)$

28. Circular Linked List with Tail Pointer

A very useful implementation maintains only a `TAIL` pointer.

Because:

```
TAIL → last node  
TAIL->next → first node
```

Therefore:

```
HEAD = TAIL->next
```

Example:



This representation makes insertion at both beginning and end efficient.

29. Complexity of Circular Linked List

Operation	Without Tail	With Tail
Traversal	$O(n)$	$O(n)$

Operation	Without Tail	With Tail
Search	$O(n)$	$O(n)$
Insert beginning	$O(1)$	$O(1)$
Insert end	$O(n)$	$O(1)$
Insert after known node	$O(1)$	$O(1)$
Delete beginning	$O(1)$	$O(1)$
Delete end	$O(n)$	$O(n)$
Delete specific value	$O(n)$	$O(n)$

30. Circular Doubly Linked List

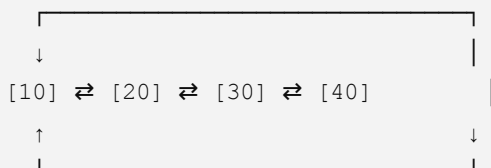
A circular list can also be **doubly linked**.

Each node contains:

PREV | DATA | NEXT

There is no `NULL`.

Example:



The relationships are:

```

HEAD->prev = TAIL
TAIL->next = HEAD
  
```

This provides:

- Forward traversal
- Backward traversal

- Efficient insertion/deletion at both ends when pointers are maintained

31. Doubly Linked vs Circular Linked List

Feature	Doubly Linked List	Circular Linked List
Links	<code>prev</code> and <code>next</code>	Usually <code>next</code>
Last node	<code>next = NULL</code>	Points to first node
First node	<code>prev = NULL</code>	Can be reached from last
Traversal	Forward and backward	Repeated circular traversal
NULL termination	Yes	No
Extra memory	More	Less for singly circular
Typical use	Browser history, undo/redo	Round-robin scheduling

32. Singly vs Doubly vs Circular Linked List

Feature	Singly	Doubly	Circular
Data field	✓	✓	✓
Next pointer	✓	✓	✓
Previous pointer	✗	✓	Optional
Forward traversal	✓	✓	✓
Backward traversal	✗	✓	Only circular doubly
Last → NULL	✓	✓	✗
Extra pointer	Low	High	Low/medium
Insert beginning	O(1)	O(1)	O(1)
Insert end*	O(n)	O(1) with tail	O(1) with tail

Feature	Singly	Doubly	Circular
Delete beginning	$O(1)$	$O(1)$	$O(1)$
Delete end*	$O(n)$	$O(1)$ with tail	$O(n)$ singly

* Complexity assumes appropriate `TAIL` pointer where indicated.

33. Applications

Doubly Linked List

Common applications include:

- Browser forward/backward navigation
- Undo/redo operations
- Music/video playlists
- LRU cache implementations
- Navigation between records

Circular Linked List

Common applications include:

- Round-robin CPU scheduling
- Circular queues
- Multiplayer turn management
- Repeated playlist systems
- Token-passing systems
- Josephus problem

34. Important Exam Points

Doubly Linked List

Remember:

```
NODE = PREV + DATA + NEXT
```

First node:

```
prev = NULL
```

Last node:

```
next = NULL
```

It supports:

Forward traversal

↓

HEAD → → → TAIL

Backward traversal

↓

TAIL → → → HEAD

Circular Linked List

Remember:

```
LAST->NEXT = HEAD
```

There is **no NULL at the end**.

Traversal must stop when:

C

```
ptr == head
```

rather than:

C

```
ptr == NULL
```

Most important complexity facts

DLL:

Insert beginning $O(1)$

Insert end with tail $O(1)$

Delete beginning $O(1)$

Delete end with tail $O(1)$

Search $O(n)$

Circular SLL:

Insert beginning $O(1)$

Insert end with tail $O(1)$

Delete beginning $O(1)$

Delete end	$O(n)$
Search	$O(n)$
Traversal	$O(n)$

Key distinction: A **doubly linked list** adds a `prev` link to enable two-way movement, while a **circular linked list** changes the termination condition so that the last node links back to the first. A list can also be **both doubly linked and circular**.